

EX:01**Design and Evaluate Different Prompt Types to Control Agent Reasoning, Autonomy, and Output Quality****Aim:**

To design and evaluate different prompt engineering techniques for controlling an AI agent's reasoning process, autonomy, and output quality, and compare their effectiveness on the same task.

Algorithm:

1. Select a core problem-solving task (designing a shortest path algorithm on graphs with Python implementation).
2. Design five distinct prompt engineering strategies to control agent behavior: Direct, Structured, Self-Check, Autonomous, and Constrained prompts.
3. Configure environment authentication with Google Gemini API key.
4. Initialize the Gemini client using the google-genai library and select the target model (gemini-3.5-flash-lite).
5. Formulate prompt variations incorporating different reasoning constraints, self-verification instructions, and autonomous agent personas.
6. Generate model responses for each prompt type using `client.models.generate_content()`.
7. Display each output with clear section headers to compare reasoning depth, correctness, and conciseness.
8. Evaluate how prompt structure guides the agent's problem-solving workflow and autonomy.
9. Record observations comparing unconstrained vs structured and self-checking agent outputs.
10. Conclude which prompt engineering techniques are best suited for controlling agent reasoning and quality.

Code:

```
import os
from google import genai

os.environ["GEMINI_API_KEY"] = ""
client = genai.Client(api_key=os.environ["GEMINI_API_KEY"])
model = "gemini-3.5-flash-lite"

task = """
Design an algorithm to find the shortest path between two nodes in a graph.
Explain the approach and provide Python code.
"""

direct_prompt = task

structured_prompt = """
Solve the following problem using a structured approach.
First understand the problem, then explain the approach,
provide the solution, and finally give the answer.

""" + task

self_check_prompt = """
Solve the following problem carefully.
After solving it, verify the solution for correctness.
Check for possible errors and provide the corrected final answer.

""" + task
```

```

autonomous_prompt = """
Act as an autonomous problem-solving agent.
Determine the steps required to solve the task and complete it independently.
Make reasonable decisions without asking for additional information.

```

```

""" + task

```

```

constrained_prompt = """
Solve the following task while following these constraints:
1. Keep the explanation concise.
2. Provide only relevant information.
3. Provide working Python code.
4. Clearly state the final solution.

```

```

""" + task

```

```

direct_response = client.models.generate_content(
    model=model,
    contents=direct_prompt
)

```

```

structured_response = client.models.generate_content(
    model=model,
    contents=structured_prompt
)

```

```

self_check_response = client.models.generate_content(
    model=model,
    contents=self_check_prompt
)

```

```

autonomous_response = client.models.generate_content(
    model=model,
    contents=autonomous_prompt
)

```

```

constrained_response = client.models.generate_content(
    model=model,
    contents=constrained_prompt
)

```

```

print("\n===== DIRECT PROMPT =====\n")
print(direct_response.text)

```

```

print("\n===== STRUCTURED PROMPT =====\n")
print(structured_response.text)

```

```

print("\n===== SELF-CHECK PROMPT =====\n")
print(self_check_response.text)

```

```

print("\n===== AUTONOMOUS PROMPT =====\n")
print(autonomous_response.text)

```

```

print("\n===== CONSTRAINED PROMPT =====\n")
print(constrained_response.text)

```

Output:

```

===== DIRECT PROMPT =====

```

To find the shortest path between two nodes in a graph, the choice of algorithm depends on the nature of the graph (weighted vs. unweighted).

Below, I present the solution using Dijkstra's Algorithm, which is the gold standard for graphs with non-negative edge weights. (If the graph has no weights, Breadth-First Search

(BFS) is typically used instead, as Dijkstra's simplifies to BFS when all weights are equal).

1. Approach Explanation (Dijkstra's Algorithm)

Dijkstra's algorithm is a greedy algorithm that finds the shortest path from a starting node to all other nodes in a graph.

How it works:

1. Initialization:

- Maintain a dictionary/array of distances from the start node to all other nodes, set to infinity, except for the start node which is set to 0.
- Maintain a priority queue (min-heap) to explore the node with the currently known shortest distance.
- Maintain a predecessor dictionary to reconstruct the final path later.

2. Exploration:

- While the priority queue is not empty, pop the node with the smallest distance (current_node).
- If current_node is the destination node, we can stop early (optimization).
- For each neighbor of current_node, calculate the tentative distance through current_node.
- If this tentative distance is shorter than the currently recorded distance to the neighbor, update the distance, record the current_node as the predecessor, and push the neighbor into the priority queue.

3. Path Reconstruction:

- Once the destination is reached, trace backward from the destination using the predecessor dictionary to build the exact shortest path.

2. Python Implementation

```
import heapq
```

```
def shortest_path(graph, start, target):
    pq = [(0, start)]
    distances = {start: 0}
    predecessors = {start: None}
    visited = set()

    while pq:
        current_distance, current_node = heapq.heappop(pq)
        if current_node == target:
            break
        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, weight in graph.get(current_node, {}).items():
            if neighbor in visited:
                continue
            tentative_distance = current_distance + weight
            if tentative_distance < distances.get(neighbor, float('inf')):
                distances[neighbor] = tentative_distance
                predecessors[neighbor] = current_node
                heapq.heappush(pq, (tentative_distance, neighbor))

    if target not in distances:
        return float('inf'), []

    path = []
    current = target
    while current is not None:
        path.append(current)
        current = predecessors[current]
```

```
path.reverse()
return distances[target], path
```

3. Complexity Analysis:

- Time Complexity: $O((V + E) \log V)$
- Space Complexity: $O(V + E)$

===== STRUCTURED PROMPT =====

1. Understand the Problem:

Find shortest path between start and target node in a weighted graph with non-negative costs.

Input: Graph adjacency list, start node, target node.

Output: Shortest distance and sequence of traversed nodes.

2. Explain the Approach:

Use Dijkstra's algorithm. Maintain a min-heap priority queue storing (distance, node) pairs. Relax edges iteratively and maintain parent pointers for backtracking.

3. Python Code:

```
import heapq

def shortest_path(graph, start, target):
    pq = [(0, start)]
    distances = {node: float('inf') for node in graph}
    distances[start] = 0
    previous_nodes = {node: None for node in graph}
    visited = set()

    while pq:
        current_distance, current_node = heapq.heappop(pq)
        if current_node == target:
            break
        if current_node in visited:
            continue
        visited.add(current_node)

        for neighbor, weight in graph[current_node].items():
            if neighbor in visited:
                continue
            distance = current_distance + weight
            if distance < distances[neighbor]:
                distances[neighbor] = distance
                previous_nodes[neighbor] = current_node
                heapq.heappush(pq, (distance, neighbor))

    path = []
    current = target
    if distances[target] == float('inf'):
        return float('inf'), []
    while current is not None:
        path.append(current)
        current = previous_nodes[current]
    path.reverse()
    return distances[target], path
```

4. Answer Summary:

Dijkstra's algorithm optimally solves single-pair shortest path on non-negative weighted graphs with $O((V + E) \log V)$ time complexity and $O(V + E)$ space.

===== SELF-CHECK PROMPT =====

1. Approach Explanation:

Dijkstra's Algorithm operates greedily using a min-heap, guaranteeing optimal shortest path for non-negative weights.

2. Python Code Implementation:

import heapq

```
def dijkstra(graph, start_node, target_node):
    distances = {node: float('inf') for node in graph}
    distances[start_node] = 0
    previous_nodes = {node: None for node in graph}
    pq = [(0, start_node)]

    while pq:
        current_distance, current_node = heapq.heappop(pq)
        if current_node == target_node:
            break
        if current_distance > distances[current_node]:
            continue
        for neighbor, weight in graph[current_node].items():
            tentative_distance = current_distance + weight
            if tentative_distance < distances[neighbor]:
                distances[neighbor] = tentative_distance
                previous_nodes[neighbor] = current_node
                heapq.heappush(pq, (tentative_distance, neighbor))

    path = []
    current = target_node
    while current is not None:
        path.insert(0, current)
        current = previous_nodes[current]
    if distances[target_node] == float('inf'):
        return float('inf'), []
    return distances[target_node], path
```

3. Verification & Correctness Check:

- Correctness: Verified optimal for non-negative weights. If negative edges exist, Bellman-Ford must be utilized.
- Time Complexity: $O((V + E) \log V)$
- Space Complexity: $O(V + E)$
- Edge Cases Handled: Start equals target returns distance 0; unreachable target gracefully returns infinity.

===== AUTONOMOUS PROMPT =====

Approach: Dijkstra's Algorithm

1. Initialization: Set start node distance to 0, all others to infinity. Initialize min-priority queue and predecessor map.
2. Exploration: Greedily extract minimum distance node, relax neighbor edges, and push updated costs into min-heap.
3. Path Reconstruction: Backtrack from target node to origin.

Complexity Analysis:

- Time Complexity: $O((V + E) \log V)$
- Space Complexity: $O(V)$ auxiliary memory

===== CONSTRAINED PROMPT =====

Approach:

For unweighted graphs, Breadth-First Search (BFS) is the most efficient shortest path algorithm, traversing level-by-level using a FIFO queue with $O(V + E)$ time complexity.

Python Code:

from collections import deque

```
def shortest_path(graph, start, goal):
    if start == goal:
        return [start]
```

```
queue = deque([start])
came_from = {start: None}
while queue:
    current = queue.popleft()
    if current == goal:
        break
    for neighbor in graph.get(current, []):
        if neighbor not in came_from:
            queue.append(neighbor)
            came_from[neighbor] = current
if goal not in came_from:
    return None
path = []
current = goal
while current is not None:
    path.append(current)
    current = came_from[current]
path.reverse()
return path
```

Final Solution:

The algorithm utilizes Breadth-First Search (BFS) achieving optimal $O(V + E)$ time and $O(V)$ space complexity.

Result:

Thus, different prompt types (Direct, Structured, Self-Check, Autonomous, and Constrained) were successfully designed, executed, and evaluated using Google Gemini API to control agent reasoning, autonomy, and output quality.